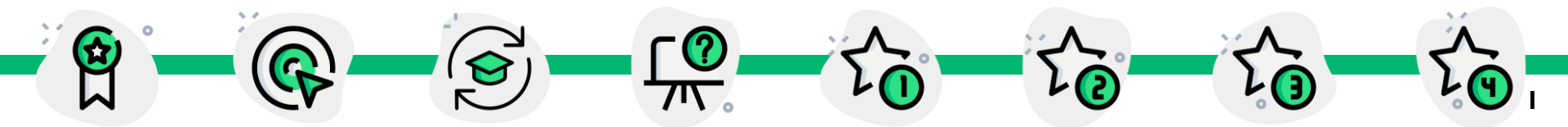


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.05.18

11주차



문자열 검색

◆ 목차

07-1 문자열의 기본

07-2 부르트-포스법

07-3 KMP법

07-4 보이어-무어법



07-1 문자열의 기본

◆ 문자열 정의하기

◆ 문자열 string

- 문자의 '나열'. 문자가 하나만 있어도 빈 문자열도 문자열

◆ 문자열 리터럴 알아보기 - (1)

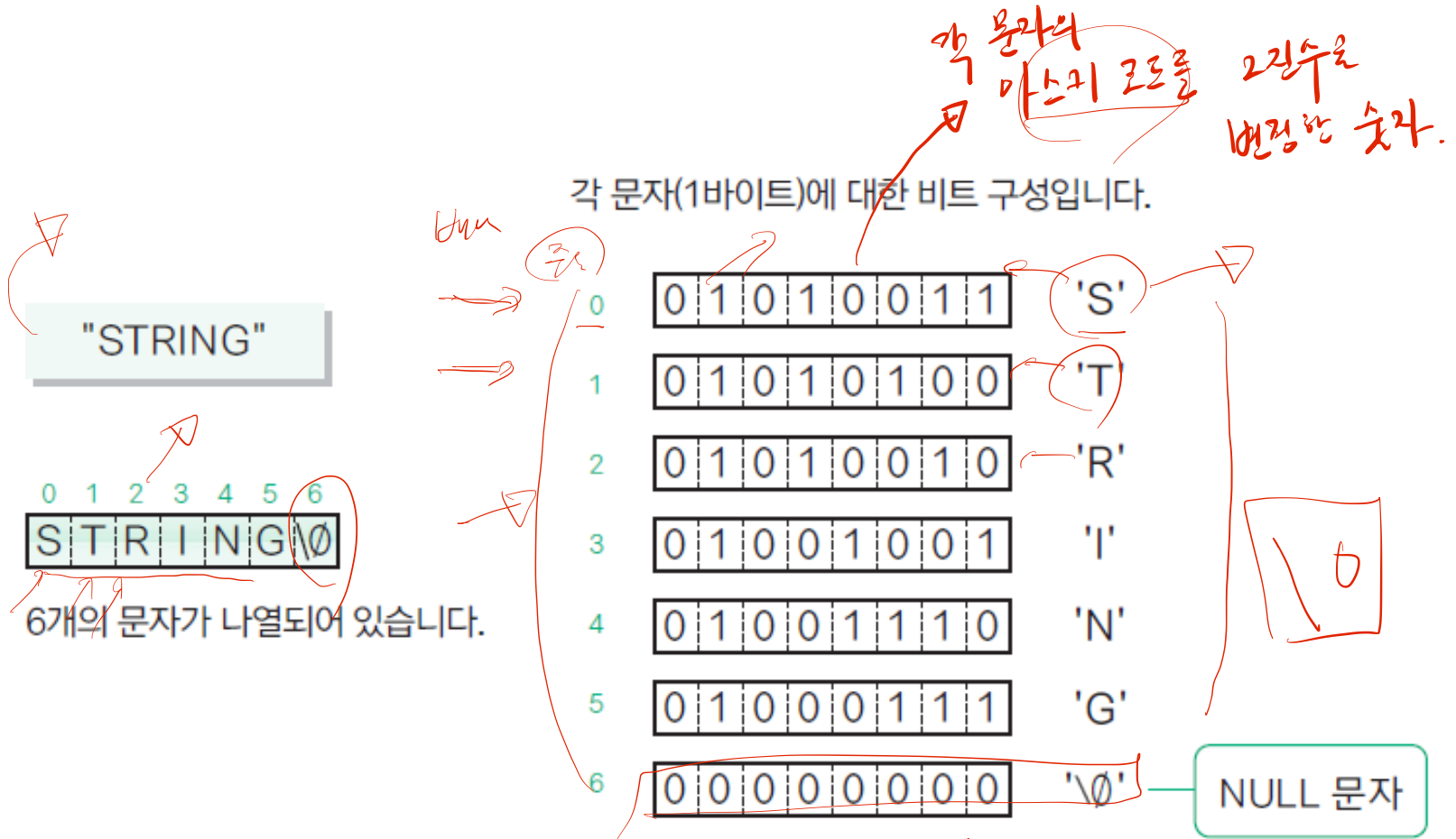
◆ 문자열 리터럴 string literal

- C 언어에서는 "STRING"이나 "ABC" 또는 그냥 "" 처럼 (0개 이상의) 문자 나열을 2개의 큰따옴표(")로 묶은 것
- 그림 7-1은 문자열 리터럴 "STRING"의 내부
- 문자열 안의 문자는 메모리 공간에 연속으로 배치
- 컴퓨터는 문자열 리터럴의 끝을 나타내기 위해 널 문자(null character)를 자동으로 추가
- 널 문자 내부는 컴퓨터 환경이나 문자 코드에 관계없이 모든 비트의 값이 0

char a = 'a'; → 2
↑
[A B C D E] → 12



07-1 문자열의 기본



[그림 7-1] 문자열 리터럴과 메모리 공간

문자열 바리막에는 NULL 문자를 포함함



07-1 문자열의 기본

◆배열에 문자열 저장하기 - (1)

◆실습 7-2

- 문자열 "ABCD"를 저장하고 해당 문자열을 출력하는 프로그램
- 문자열 값을 char형 배열에 저장하면 자유롭게 값을 꺼내고 쓸 수 있음
- 요소의 자료형이 char형이고 요소의 개수가 8인 char[8]형 배열 st의 요소에 문자열을 구성할 문자를 하나씩 순서대로 대입
 - 대입한 후 배열 st의 모습은 그림 7-2와 같음
- st[4]에 대입된 널 문자는 문자열의 끝을 나타냄
- st[5]부터 어떤 문자를 대입해도 널 문자까지만 문자열이라고 인식



07-1 문자열의 기본

◆ 실습 7-2

Chapt07_실습 7-2/str_ary1.c

// 문자열을 배열로 저장해서 출력

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
char st[8];
```

```
st[0] = 'A';
```

```
st[1] = 'B';
```

```
st[2] = 'C';
```

```
st[3] = 'D';
```

```
st[4] = '\0';
```

```
printf("문자열 st에는 \"%s\"가 들어 있습니다.\n", st);
```

```
return 0;
```

```
}
```

st

문자열을 배열로 저장



int a = 1

1



07-1 문자열의 기본

◆배열에 문자열 저장하기 - (2)

◆문자열 초기화

- 문자열을 선언하면서 동시에 초기화할 수 있음
- 배열 st를 초기화하며 선언하는 코드
 - 보통은 두 번째의 간단한 선언 방식을 많이 사용
- 요소 개수를 생략하고 선언하면 초기화할 문자열의 저장에 필요한 최소한의 요소 개수로 배열이 만들어짐
- 주의할 점은 앞에서 초기화할 때 사용한 코드는 초기화에서만 사용해야 함

① `char st[10] = {'A', 'B', 'C', 'D', '\0'};` → 방법 ①

② `char st[10] = "ABCD";` → 방법 ②

③ `char st[] = "ABCD";` // 배열 st의 요소 개수는 5 → 방법 ③

`char st[10];`
`st = {'A', 'B', 'C', 'D', '\0'};` // 오류
`st = "ABCD";` // 오류



07-1 문자열의 기본

◆ 실습 7-3

Chapt07_실습 7-3/str_ary2.c

// 문자열을 배열에 초기화하며 선언

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
char st[8] = "ABCD";
```

```
printf("문자열 st에는 \"%s\"가 들어 있습니다.\n", st);
```

```
return 0;
```

```
}
```

1) printf의 st를 배열 형태로 변경해보라.

2) 문자열 st에는 "A"가 들어있습니다 를 출력하도록 손코딩해보라



07-1 문자열의 기본

◆포인터로 문자열 나타내기 - (1)

◆실습 7-4

- 문자열을 출력하는 프로그램(포인터를 사용) 
- 선언한 char *형 포인터 pt는 초기화 요소로 문자열 리터럴 "12345"를 줌
- 문자열 리터럴을 평가하여 얻는 값은 그 문자열 리터럴의 첫 번째 문자에 대한 포인터이므로 포인터 pt는 그림 7-3에 나타낸 것처럼 "12345"가 들어 있는 메모리 영역의 첫 번째 문자 '1'을 가리키며 초기화
- 포인터 pt는 마치 배열인 것처럼 행동
- 문자열 리터럴 "12345"의 문자 '1', '2', '3', '4', '5'는 순서대로 pt[0], pt[1], pt[2], pt[3], pt[4]의 인덱스 식으로 접근할 수 있음



07-1 문자열의 기본

◆ 실습 7-4

Chapt07_실습 7-4/str_ptr.c

// 문자열을 출력(포인터를 사용)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
char* pt = "12345";
```

```
printf("포인터 pt는 \"%s\" 를(을) 가리킵니다.\n", pt);
```

```
return 0;
```

```
}
```

✱ pt ⇒ 문자열의 첫 주소값을 저장

✱ 문자열이 아닌 주소값을 출력하려면

코드를 어떻게 수정해야 할까? %p

✱ printf("%d", xpt) ? 어떻게 출력?
⇒ 49

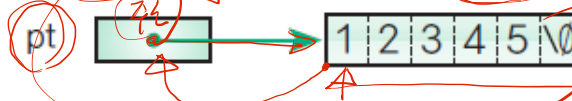
▷ 문자열의 시작 주소를

포인터 pt에 저장!!

▷ 이 주소부터 시작해서 \0 까지
출력하기

```
char *pt = "12345";
```

포인터 pt는 문자열 리터럴 "12345"의 첫 번째 문자 '1'을 가리킵니다.



[그림 7-3] 포인터와 문자열



07-1 문자열의 기본

arr [A | B | C | D | \0]

pt [32] → [A | B | C | D | \0]

◆ 포인터로 문자열 나타내기 - (2)

◆ 배열에 의한 문자열과 포인터에 의한 문자열이 갖는 메모리 영역의 크기 비교

- 배열에 저장한 문자열은 6바이트의 메모리 영역을 가짐
- 포인터로 표현한 문자열은 문자열 리터럴을 저장하기 위한 영역 외에도 pt가 갖는 메모리 영역이 더 필요함
- 즉, pt를 위한 영역인 `sizeof(char *)` 바이트와 또 문자열 리터럴을 위한 영역인 `sizeof("12345")` 바이트만큼 메모리 영역이 필요함
- 다시 말해 포인터로 표현한 문자열은 배열에 저장한 문자열보다 더 많은 메모리 영역을 차지



07-1 문자열의 기본

*p = 4

int a = 3;

int *p; → 정언어

p = &a;
주소 주소

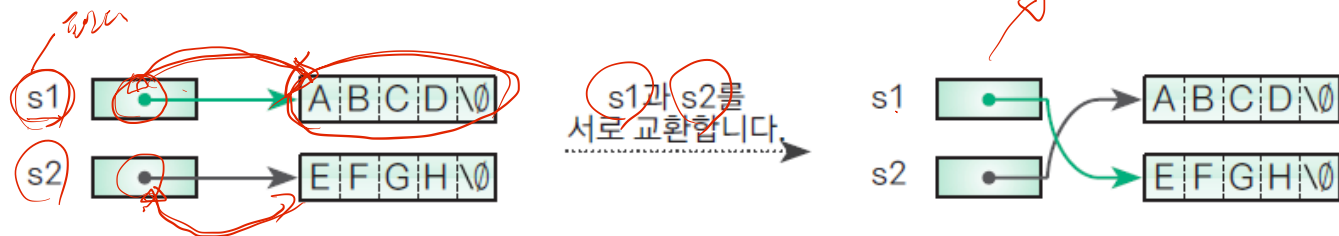
```
A (int *p)
{
}

main() {
    int a = 3;
    A(&a);
}
```

◆포인터로 문자열 나타내기 - (3)

◆실습 7-5

- 문자열 리터럴을 가리키는 두 포인터의 값을 서로 교환하는 프로그램
- main 함수에서 2개의 포인터 s1과 s2를 선언한 다음 포인터 s1은 "ABCD", 포인터 s2는 "EFGH"를 가리키며 초기화
- swap_ptr 함수는 2개의 포인터값을 서로 교환
- 이 함수가 전달받는 매개변수 x, y의 자료형은 포인터(s1, s2)의 주소를 받아야 하기 때문에 char **형으로 정의(포인터를 가리키는 포인터)



[그림 7-4] 두 포인터를 교환하는 과정



07-1 문자열의 기본

◆ 실습 7-5

Chapt07_실습 7-5/swap_ptr.c

// 두 포인터값을 서로 교환하는 함수가 있는 프로그램

#include <stdio.h>

/*--- 두 포인터를 서로 교환하는 함수 ---*/

void swap_ptr(char** x, char** y)

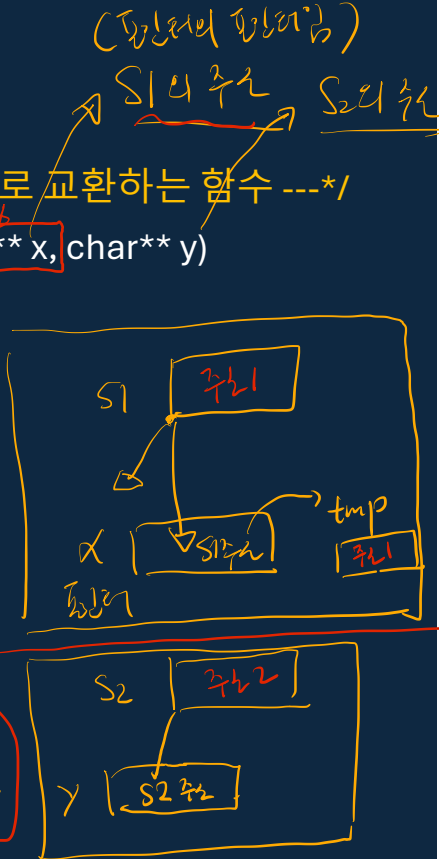
{

char* tmp = *x;

*x = *y;

*y = tmp;

}



∴ *x = s2
*y = s1
↓
주소값을 바꾼다

int main(void)

{

char* s1 = "ABCD"; // s1은 "ABCD"의 첫 글자 'A'를 가리킴

char* s2 = "EFGH"; // s2는 "EFGH"의 첫 글자 'E'를 가리킴

printf("포인터 s1은 \"%s\"를 가리킵니다.\n", s1);

printf("포인터 s2는 \"%s\"를 가리킵니다.\n", s2);

swap_ptr(&s1, &s2);

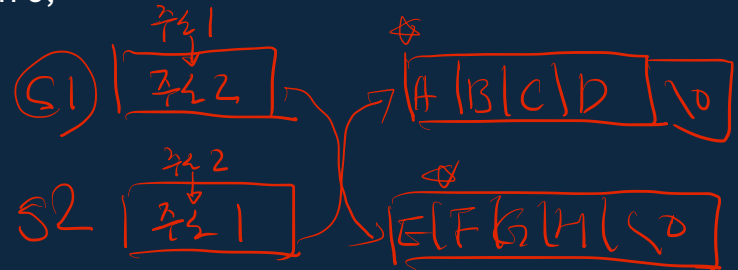
puts("\n포인터 s1과 s2의 값을 서로 교환했습니다.\n");

printf("포인터 s1은 \"%s\"를 가리킵니다.\n", s1);

printf("포인터 s2는 \"%s\"를 가리킵니다.\n", s2);

return 0;

}



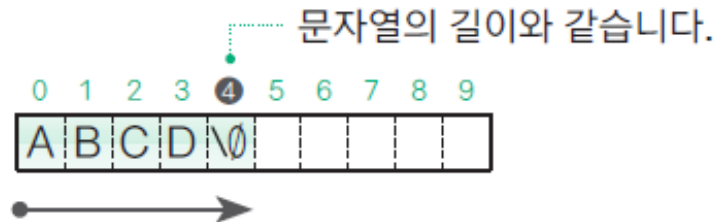


07-1 문자열의 기본

◆ 문자열의 길이 구하기 - (1)

◆ 문자열의 길이를 구하는 알고리즘

- 문자열의 첫 문자부터 널 문자까지 선형 검색을 하면 됨
- 이때 문자열의 끝은 널 문자이므로 찾은 널 문자의 인덱스는 문자열의 길이와 같음



널 문자를 선형 검색합니다.

[그림 7-5] 문자열의 길이를 구하는 과정



07-1 문자열의 기본

◆ 문자열의 길이 구하기 - (2)

◆ 실습 7-6

- 문자열의 길이를 구하는 프로그램
- `str_len` 함수는 전달받은 문자열 `s`의 길이를 구하는 함수
- `while` 문은 문자열을 앞에서부터 차례로 스캔
- `s[len]`이 0이 아니면 반복해 같은 작업을 수행
- 널 문자를 만나면 `s[len]`가 0이 되어 문자열 스캔을 중단
- `while` 문을 빠져나온 후 `len`의 값은 널 문자가 들어 있는 요소의 인덱스와 같으며 다시 말해, 문자열의 길이가 됨
- 문자열의 끝에는 반드시 널 문자가 있어 검색에 실패하는 경우는 없음
- 문자열의 길이를 구하는 알고리즘은 검색에 실패할 가능성을 고려할 필요가 없는 '보초법에 의한 선형 검색'



07-1 문자열의 기본

◆ 실습 7-6

Chapt07_실습 7-6/str_len1.c

// 문자열의 길이를 구하는 프로그램

```
#include <stdio.h>
```

```
/*--- 문자열 s의 길이를 구하는 함수(버전 1) ---*/
```

```
int str_len(const char* s)
```

```
{
```

```
    int len = 0;
```

```
    while (s[len])
```

```
        len++;
```

```
    return len;
```

```
}
```

배열의 첫 번째 입력

→ s[len]

1) s[0] → 'a' (len=0)

len=1

2) s[1] → 'b' (len=1)

len=2

3) s[2] → 'c' (len=2)

4) s[3] → '\0' (len=3)

```
int main(void)
```

```
{
```

```
    char str[256];
```

```
    printf("문자열: ");
```

```
    scanf("%s", str);
```

```
    printf("이 문자열의 길이는 %d입니다.\n", str_len(str));
```

```
    return 0;
```

```
}
```

str

배열의 첫 번째 입력

str[2] = [a][b][c][\0]

str

3



07-1 문자열의 기본

◆ 문자열의 길이 구하기 - (3)

◆ 실습 7-7과 실습 7-8

- `str_len` 함수를 각각 다른 방법으로 구현한 프로그램



07-1 문자열의 기본

◆ 실습 7-7, 8

Chapt07_실습 7-5/ str_len2.c

// 문자열 s의 길이를 구하는 함수(버전 2)

#include <stdio.h>

/*--- 문자열 s의 길이를 구하는 함수(버전 2) ---*/

int str_len(const char* s)

```
{
    int len = 0;
    while (*s++)
        len++;
    return len;
}
```

str_len(str)

① while (*s++)
↓
실행 후 len=1, *s = *(s+1)
↘ 'A'를 의미

② while (*(s+1)++)
↓
실행 후 len=2, *(s+1) = *(s+2)
↘ 'B'를 의미

③ while (*(s+2)++)
↓
실행 후 len=3, *(s+2) = *(s+3)
↘ 'C'를 의미

∴ len=3을 반환 (정답) ← \0은 4번째

Chapt07_실습 7-5/ str_len3.c

// 문자열 s의 길이를 구하는 함수(버전 3)

#include <stdio.h>

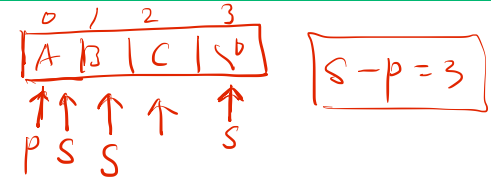
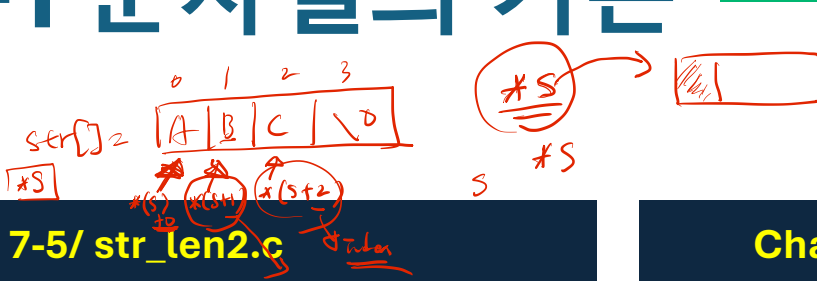
/*--- 문자열 s의 길이를 구하는 함수(버전 3) ---*/

int str_len(const char* s)

```
{
    const char* p = s;
    while (*s)
        s++;
    return s - p;
}
```

S → 배열의 첫 주소를 저장
→ 처음 위치를 p에 저장
→ 첫 주소에 저장된 값은 의미 없음

→ *(s+0) → 첫 번째 글자 : A
→ *(s+1) → 두 번째 글자 : B
→ *(s+2) → 세 번째 글자 : C
→ *(s+3) → 네 번째 글자 : \0





07-1 문자열의 기본



◆ 문자열의 길이 구하기 - (4)

◆ strlen 함수

- C 언어 표준 라이브러리에는 문자열의 길이를 구하는 strlen 함수가 있음
- 이 함수의 반환값은 size_t형(int형이 아니라는 점에 주의)

strlen 함수	
헤더	#include <string.h>
형식	size_t strlen(const char *s);
해설	s가 가리키는 문자열의 길이를 구합니다.
반환값	구한 문자열의 길이를 반환합니다.



07-1 문자열의 기본

◆ 문자열에서 문자 검색하기 - (1)

◆ 실습 7-9 - (1)

- `str_chr` 함수는 문자열 `s`에서 문자 `c`를 선형 검색하고 검색에 성공하면 찾은 문자의 인덱스를 반환
- 이때 같은 문자가 여러 개 있는 경우 가장 앞쪽에 있는 문자의 인덱스를 반환
- 검색에 실패할 경우 `-1`을 반환



07-1 문자열의 기본

Do it! 실습 7-9

• 완성 파일 chap07/str_chr.c

01 /* 문자열에서 문자를 검색하는 프로그램 */

02 #include <stdio.h>

03
04 /*--- 문자열 s에서 문자 c를 검색하는 함수 ---*/

05 int str_chr (const char *s, int c)

06 {
07 int i = 0;

08 c = (char)c;

09 while(s[i] != c) {

10 if(s[i] == '\0') {

11 return -1;

12 i++;

13 }

14 return i;

15 }

실행 결과

문자열: SURROUND

검색할 문자: R

문자 'R'은(는) 3번째에 있습니다.

0 1 2 3 4 5 6 7 8
S U R R O U N D \0

문자 'R'을 선형 검색합니다.

0 1 2 3 4 5 6 7 8 9
S U R R O U N D \0



[그림 7-6] 문자열에서 문자를 검색하는 과정



07-1 문자열의 기본

```
16 int main (void)
17 {
18     char str[128];           // 이 문자열에서 검색
19     char tmp[128];
20     int ch;                  // 검색할 문자
21     int idx;
22
23     printf("문자열: ");
24     scanf("%s", str);        문자열은 입력받아 str 배열에 저장
25
26     printf("검색할 문자: ");
27     scanf("%s", tmp);        문자를 입력받아 tmp 배열에 저장
28     ch = tmp[0];             // 먼저 문자열로 검색할 문자를 읽음
29                             // 첫 번째 문자를 검색할 문자로 지정
30     if((idx = str_chr(str, ch)) == -1) // 처음 나오는 문자를 검색
31         printf("문자 '%c'은(는) 문자열에 없습니다.\n", ch);
32     else
33         printf("문자 '%c'은(는) %d번째에 있습니다.\n", ch, idx + 1);
34
35     return 0;
36 }
```



07-1 문자열의 기본

◆ 문자열에서 문자 검색하기 - (3)

◆ strchr 함수와 strrchr 함수

- C 언어 표준 라이브러리는 문자열 안에 들어 있는 문자를 검색하는 함수로, strchr 함수와 strrchr 함수를 제공
- 검색할 문자가 문자열 안에 여러 개 있는 경우 strchr 함수는 가장 앞쪽의 문자를 검색하지만 strrchr 함수는 가장 뒤쪽의 문자를 찾음
- 이 함수는 찾은 문자의 인덱스가 아니라 그 문자에 대한 포인터를 반환(문자의 인덱스를 반환하는 게 아니라는 점을 명심)
- 검색에 실패할 경우에는 널 포인터 NULL을 반환
- 주의할 점은 앞에서 살펴본 str_chr 함수와 strchr, strrchr 함수는 모두 두 번째 인수를 char형이 아닌 int형으로 받고 있다는 점
- 표준 라이브러리 함수에서 '문자'를 주고받을 때는 char형이 아니라 int형으로 주고받음



07-1 문자열의 기본

strchr 함수

헤더	#include <string.h>
형식	char *strchr(const char *s, int c);
해설	s가 가리키는 문자열에서 가장 앞에 있는 c를 찾습니다. 이때 c는 널 문자여도 됩니다.
반환값	찾은 문자에 대한 포인터를 반환합니다. 문자가 없으면 널 포인터를 반환합니다.

strrchr 함수

헤더	#include <string.h>
형식	char *strrchr(const char *s, int c);
해설	s가 가리키는 문자열 가운데 가장 뒤쪽에 있는 c를 찾습니다. 이때 c는 널 문자여도 됩니다.
반환값	찾은 문자에 대한 포인터를 반환합니다. 문자가 없으면 널 포인터를 반환합니다.

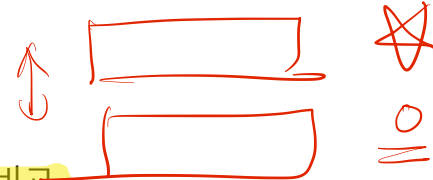


07-1 문자열의 기본

◆ 문자열의 대소 관계 비교하기 - (1)

문자열 비교 함수

◆ strcmp 함수



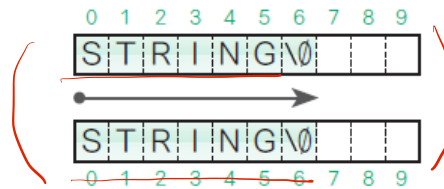
- 이 함수는 그림 7-7과 같이 두 문자열을 처음부터 순서대로 비교
- 'a'와 같이 모든 문자가 일치하면 0을 반환
- 'b'와 같이 문자를 비교하는 도중에 하나의 문자라도 다르면 두 문자열은 일치하지 않음을 알 수 있으므로 0이 아닌 값을 반환
- strcmp 함수는 문자 코드 체계에 따라 문자열의 대소 관계를 비교하고 판단 결과를 반환
 - 예를 들어 'A'가 '1'보다 큰지 혹은 'A'가 'a'보다 큰지와 같은 판단 기준은 문자 코드 체계에 따라 달라짐
- 뿐만 아니라 strcmp 함수가 반환하는 값도 컴퓨터 환경에 종속적
 - 예를 들어, 'N'이 'K'보다 큰 경우 1을 반환하는 컴퓨터 환경도 있고 문자 코드 값의 차이인 'N' - 'K'의 값을 그대로 계산하여 반환하는 컴퓨터 환경도 있음



07-1 문자열의 기본

strcmp 함수	
헤더	#include <string.h>
형식	int strcmp(const char *s1, const char *s2);
해설	s1, s2가 가리키는 문자열의 대소 관계를 비교합니다. 처음부터 순서대로 한 문자씩 unsigned char형 값으로 비교합니다.
반환값	문자열이 같으면 0, s1이 s2보다 크면 양의 정수, 작으면 음의 정수값을 반환합니다.

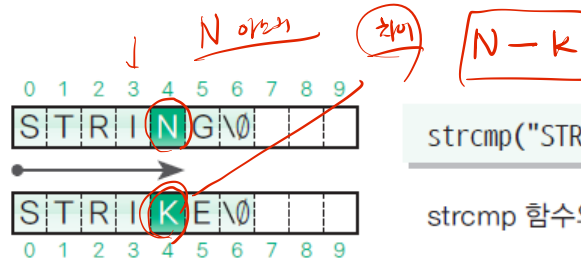
a 문자열이 일치하는 경우



```
strcmp("STRING", "STRING");
```

strcmp 함수의 반환값은 0입니다.

b 문자열이 일치하지 않는 경우



```
strcmp("STRING", "STRIKE");
```

strcmp 함수의 반환값은 0이 아닌 값입니다.

[그림 7-7] strcmp 함수를 사용한 문자열의 대소 관계 비교



07-1 문자열의 기본

◆ 문자열의 대소 관계 비교하기 - (2)

◆ 실습 7-10

- strcmp 함수와 동일하게 동작하는 문자열 비교 함수를 작성한 프로그램
- 이 프로그램은 문자열 "ABCD"와 입력한 문자열을 비교
- strcmp 함수와 마찬가지로 두 문자열이 다를 때 위의 str_cmp 함수가 반환하는 값은 문자 코드 체계에 따라 다름



07-1 문자열의 기본

◆ 실습 7-10

Chapt07_실습 7-10/ str_cmp.c

// 문자열을 비교하는 프로그램

#include <stdio.h>

/*--- 두 문자열 s1과 s2를 비교하는 함수 ---*/

int str_cmp(const char* s1, const char* s2)

{

while (*s1 == *s2) {

if (*s1 == '\0') // 같음

return 0;

s1++;

s2++;

}

return (unsigned char)*s1 - (unsigned char)*s2;

}

int main(void)

{

char st[128];

puts("\nABCD\와 비교합니다.");

puts("\nXXXX\면 종료합니다.");

while (1) {

printf("문자열 st: ");

scanf("%s", st);

if (str_cmp("XXXX", st) == 0)

break;

printf("str_cmp(\"ABCD\", st) = %d\n", str_cmp("ABCD", st));

}

return 0;

}

비교대상

비교하려는 값

실행 결과

"ABCD"와 비교합니다.

"XXXX"면 종료합니다.

문자열 st: AX

str_cmp("ABCD", st) = -22

문자열 st: AA

str_cmp("ABCD", st) = 1

문자열 st: ABCD

str_cmp("ABCD", st) = 0

문자열 st: XXXX

// 문자열을 비교하는 프로그램

```
#include <stdio.h>
```

```
/*--- 두 문자열 s1과 s2를 비교하는 함수 ---*/
```

```
int str_cmp(const char* s1, const char* s2)
{
    while (*s1 == *s2) {
        if (*s1 == '\0') // 같음
            return 0;
        s1++;
        s2++;
    }
    return (unsigned char)*s1 - (unsigned char)*s2;
}
```

s1 = 'A B C D'

s1 | 첫자리
'A B C D' (비교대상)

s2 | 첫자리
'A X' (비교대상)

while
① (*s1 == *s2) → 포인터 s1이 가리키는 주소
 ↓ ↓
 'A' 'A'
 26 26
 [" s2가 " 값과 비교해서 같은가?]

if (*s1 == '\0') → 마지막 문자인지 확인
 ↑
 null

↓
s1++ → (s1+1)
s2++ → (s2+1)

② while (*s1 == *s2)
 ↑
 null

↓

s1
↓
[A | B | C | D | \0]

[A | X | \0 | |]
 ↑
 s2

return (unsigned char)*s1 - (unsigned char)*s2
 ↓ ↓
 B (66) X (88)
 [66 - 88 = -22]



07-1 문자열의 기본

◆ 문자열의 대소 관계 비교하기 - (3)

◆ strcmp 함수

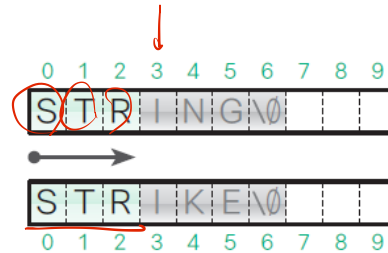
- 이 함수도 strcmp 함수와 마찬가지로 두 개 문자열의 대소 관계를 비교
- strcmp 함수는 문자열의 길이를 지정하여 비교할 수 없었지만 strncmp 함수는 3번째 인수로 지정한 문자열의 길이만큼만 비교할 수 있음
- 널 문자가 없는 '문자 배열' 간의 비교도 가능
- `a`는 처음 3개의 문자를 비교한 경우
- `b`는 처음 5개의 문자를 비교한 경우



07-1 문자열의 기본

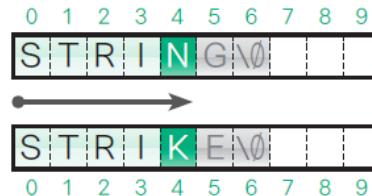
strncmp 함수	
헤더	#include <string.h>
형식	int strncmp(const char *s1, const char *s2 size_t n);
해설	s1, s2가 가리키는 문자 배열에서 n번째 문자까지의 대소 관계를 비교합니다. 처음부터 순서대로 한 문자씩 unsigned char형 값으로 비교합니다. 널 문자 이후의 비교는 하지 않습니다.
반환값	문자 배열이 같으면 0, s1이 s2보다 크면 양의 정숫값, 작으면 음의 정숫값을 반환합니다.

a 문자열이 일치하는 경우



`strncmp("STRING", "STRIKE", 3);`
strncmp 함수의 반환값은 0입니다.

b 문자열이 일치하지 않는 경우



`strncmp("STRING", "STRIKE", 5);`
strncmp 함수의 반환값은 0이 아닌 값입니다.

[그림 7-8] strncmp 함수를 사용한 문자열의 대소 관계 비교



07-1 문자열의 기본

◆ 문자열의 대소 관계 비교하기 - (4)

◆ 실습 7-11

- `strncmp` 함수를 사용한 프로그램
- 이 프로그램은 문자열 "STRING"과 배열 `st`로 읽은 문자열에서 처음부터 3개의 문자를 비교한 결과를 출력



07-1 문자열의 기본

◆ 실습 7-11

Chapt07_실습 7-11/strncmp_test.c

// 문자열을 비교하는 프로그램(strncmp 함수 사용)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(void)
```

```
{
```

```
    char st[128];
```

```
    puts("\nSTRING\의 처음 3개의 문자와 비교합니다.");
```

```
    puts("\nXXXX\를 입력하면 종료합니다.");
```

```
    while (1) {
```

```
        printf("문자열 st: ");
```

```
        scanf("%s", st);
```

```
        if (strncmp("XXXX", st, 3) == 0)
```

```
            break;
```

```
        printf("strncmp(\"STRING\", st, 3) = %d\n", strncmp("STRING", st, 3));
```

```
    }
```

```
    return 0;
```

```
}
```

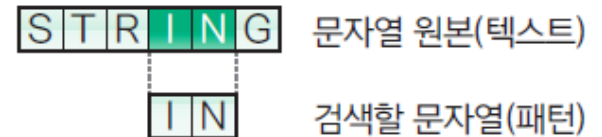


07-2 브루트-포스법

◆ 문자열 검색 정의하기

◆ 문자열 검색 string searching 알고리즘

- 문자열 검색이란 어떤 문자열 안에 다른 문자열이 들어 있는지 조사하고 들어 있다면 그 위치를 찾아내는 것
- 이해하기 쉽게 검색할 문자열을 패턴(pattern)이라 하고 문자열 원본을 텍스트(text)라고 함



해당 문자열 패턴(IN)을 문자열 원본(String)에서 검색합니다.

[그림 7-9] 문자열 검색

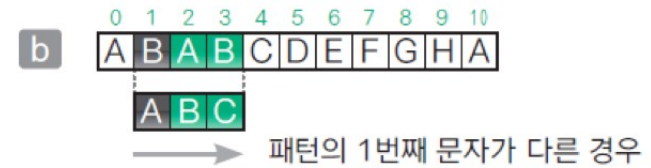
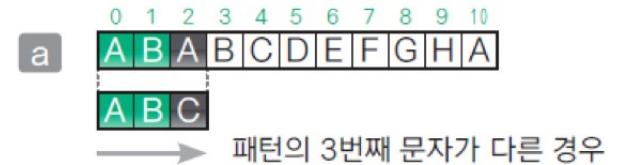


07-2 브루트-포스법

◆ 브루트-포스법으로 검색하기 - (1)

◆ 브루트-포스법 - (1)

- a 텍스트의 첫 문자 'A'부터 시작하는 3개의 문자와 " ABC " 가 일치하는지 검사
 - 'A'와 'B'는 일치하고 'C'는 다름
- b 패턴을 1칸 뒤로 옮기고 텍스트의 2번째 문자부터 3개의 문자가 일치하는지 조사
 - 'A'와 'B'가 다름
- c 패턴을 다시 1칸 뒤로 옮김
 - 'A', 'B', 'C' 모두 일치



[그림 7-10] 브루트-포스법

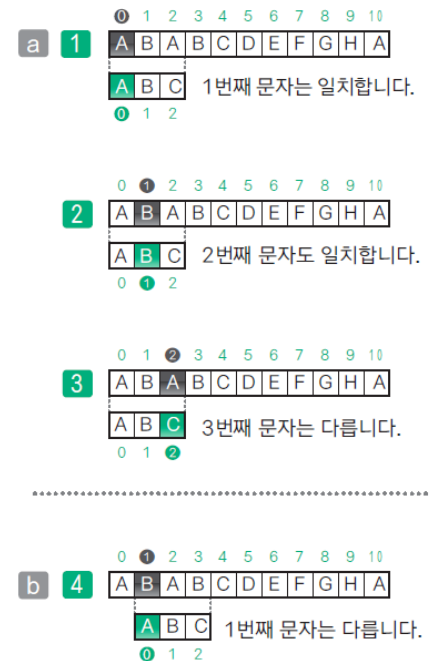


07-2 브루트-포스법

◆ 브루트-포스법으로 검색하기 - (2)

◆ 브루트 포스법 - (2)

- 선형 검색을 확장한 알고리즘이므로 단순법(單純法), 소박법(素朴法)이라고도 함
- a 텍스트의 ① 문자와 패턴의 ① 문자가 겹치도록 두 줄로 놓고 1번째 문자부터 순서대로 일치하는지 검사
 - 1 2 처럼 문자가 일치하면 계속해서 패턴과 텍스트의 문자를 검사
 - 그러다가 3 처럼 다른 문자가 나타나면 검사를 중단
- b 검사할 텍스트의 위치를 1칸 뒤로 이동해서 텍스트의 ① 문자와 패턴의 ① 문자를 겹치도록 두 줄로 놓음
 - 4 에서 볼 수 있듯이 1번째 문자부터 다르므로 검사를 중단



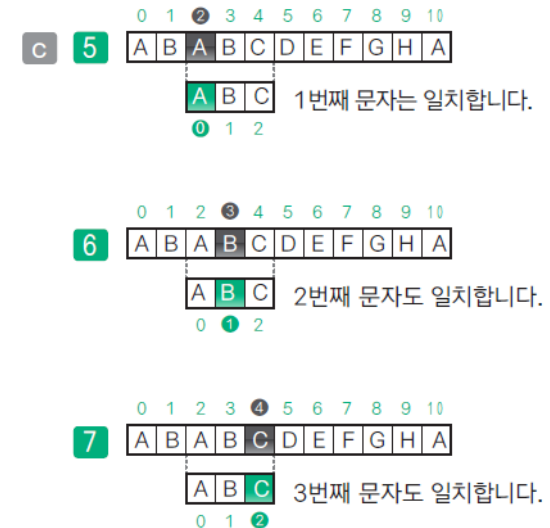


07-2 브루트-포스법

◆ 브루트-포스법으로 검색하기 - (3)

◆ 브루트 포스법 - (2)

- **c** 조사할 텍스트의 위치를 1칸 뒤로 이동해서 텍스트의 **2** 문자와 패턴의 **0** 문자를 겹치도록 두 줄로 놓음
 - **5** **6** **7** 에서 볼 수 있듯이 모든 문자가 일치하므로 검색 성공
- 이때 **3** 에서 텍스트의 검사 위치를 **2** 까지 진행시켜도 **4** 에서는 다시 **1** 부터 검사
- 즉, 이미 검사를 진행한 위치를 기억하지 못하므로 브루트-포스법의 효율은 좋지 않다고 할 수 있음
- 패턴을 하나씩 옮기다가 패턴의 끝부분이 텍스트의 끝부분을 벗어나면 검색 실패



[그림 7-11] 브루트-포스법으로 검색하는 과정

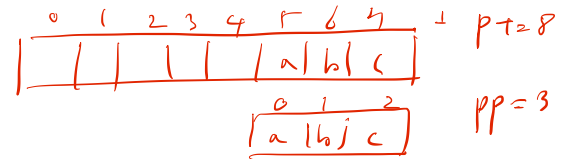
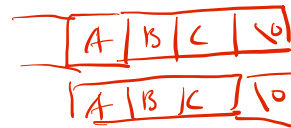


07-2 브루트-포스법

◆ 브루트-포스법으로 검색하기 - (4)

◆ 실습 7-12 - (1)

- bf_match 함수는 텍스트(txt)에서 패턴(pat)을 검색하여 텍스트의 위치(인덱스)를 반환
- 텍스트에 패턴이 여러 개 있는 경우에 가장 앞에 위치한 텍스트의 인덱스를 반환
- 검색에 실패하면 -1을 반환
- 텍스트(txt)를 스캔하기 위한 변수로 pt를 사용하고, 패턴(pat)을 스캔하기 위한 변수로 pp를 사용
- 두 변수는 처음에 0으로 초기화하고 스캔을 하거나 패턴을 옮길 때마다 업데이트



Do it! 실습 7-12

• 완성 파일 chap07/bf_match.c

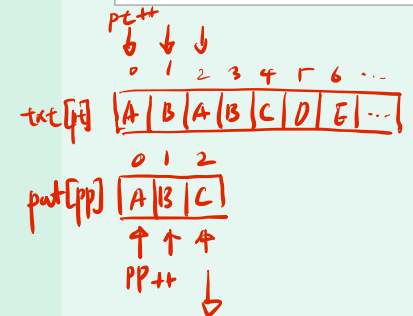
```

01 // 브루트-포스법으로 문자열을 검색하는 프로그램
02 #include <stdio.h>
03
04 /*--- 브루트-포스법으로 문자열을 검색하는 함수 ---*/
05 int bf_match (const char txt[], const char pat[])
06 {
07     int pt = 0; // txt 커서
08     int pp = 0; // pat 커서
09     while(txt[pt] != '\0' && pat[pp] != '\0') {
10         if(txt[pt] == pat[pp]) {
11             pt++;
12             pp++;
13         } else {
14             pt = pt - pp + 1;
15             pp = 0;
16         }
17     }
18     if(pat[pp] == '\0')
19         return pt - pp;
20     return -1;
21 }

```

실행 결과

브루트-포스법
텍스트: ABABCEFGHA
패턴: ABC
3번째 문자부터 match합니다.



- ① $pt = pt - pp + 1$
 $\therefore pt = 1 \rightarrow \text{txt}[1] \text{ 배열 1번으로}$
 $pp = 0 \rightarrow \text{pat}[0] \text{ 배열 0번으로}$
- ② $pt = pt - pp + 1$
 $= 1 - 0 + 1 = 2$
 $\therefore pt = 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
 $pp = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$
- ③ $pt - pp : \text{결과} (5-3)$



07-2 브루트-포스법

◆브루트-포스법으로 검색하기 - (5)

◆실습 7-12 - (2)

- 브루트-포스법으로 문자열을 검색하는 프로그램

```
23 int main (void)
24 {
25     char s1[256];    // 텍스트
26     char s2[256];    // 패턴
27     puts("브루트-포스법");
28     printf("텍스트: ");
```

```
29     scanf("%s", s1);
30     printf("패턴: ");
31     scanf("%s", s2);
32     int idx = bf_match(s1, s2); // 텍스트(s1)에서 패턴(s2)을 브루트-포스법으로 검색
33     if(idx == -1)
34         puts("텍스트에 패턴이 없습니다.");
35     else
36         printf("%d번째 문자부터 match합니다.\n", idx + 1);
37
38     return 0;
39 }
```

→ 위치는 인덱스 + 1로
실제 위치



07-3 KMP 법

◆ KMP법 알아보기 - (1)

◆ KMP법 - (1)

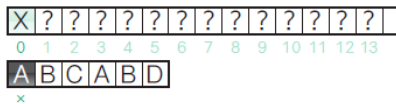
- KMP법은 검사했던 위치 결과를 버리지 않고 이를 효율적으로 활용하는 알고리즘
- KMP법은 텍스트와 패턴의 겹치는 부분을 찾아내어 검사를 다시 시작할 위치를 구함
- '몇 번째 문자부터 다시 검색할지'에 대한 값을 그림 7-12처럼 미리 '표'로 만듦
 - a ~ d 패턴의 1 ~ 4번째 문자에서 검사에 실패한 경우에는 패턴을 옮긴 다음 1번째 문자부터 다시 검사
 - e 패턴의 5번째 문자에서 검사에 실패한 경우에는 패턴을 옮긴 다음 1번째 문자가 일치하므로 2번째 문자부터 다시 검사할 수 있음
 - f 패턴의 6번째 문자에서 검사에 실패한 경우에는 3번째 문자부터 다시 검사할 수 있음



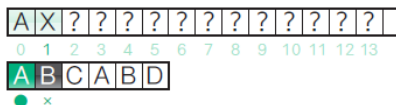
07-3 KMP 법

a 1번째 문자가 일치하지 않습니다.

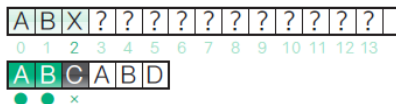
A	B	C	A	B	D
-	0	0	1	2	0



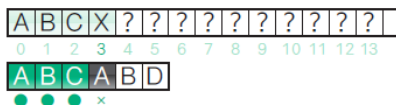
b 2번째 문자가 일치하지 않습니다.



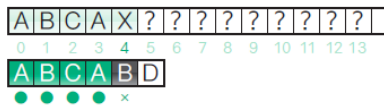
c 3번째 문자가 일치하지 않습니다.



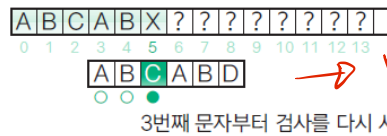
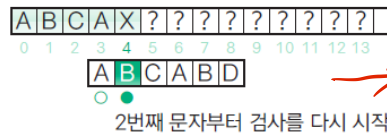
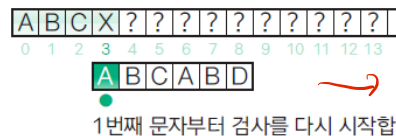
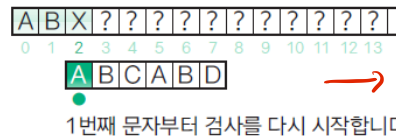
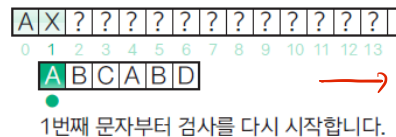
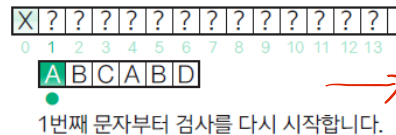
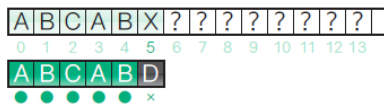
d 4번째 문자가 일치하지 않습니다.



e 5번째 문자가 일치하지 않습니다.



f 6번째 문자가 일치하지 않습니다.



[그림 7-12] 각 단계에서 검사를 다시 시작할 위치의 값



07-3 KMP 법

◆ KMP법 알아보기 - (2)

◆ KMP법 - (2)

- 표를 작성할 때는 패턴 안에서 중복되는 문자의 나열을 먼저 찾아야 함
- 패턴 "ABCABD"를 1칸 뒤로 옮긴 다음 겹침
 - 초록색 부분이 겹치지 않으므로 패턴을 옮긴 다음 앞쪽의 1번째 문자부터 검사를 다시 시작해야 한다는 것을 알 수 있음
 - 따라서 표에서 2번째 문자(B)의 값을 0으로 함
 - 값이 0인 이유는 아래에 위치시킨 패턴의 첫 번째 문자의 인덱스가 0이고 이 위치에서 다시 검사를 시작하기 때문
- 패턴을 1칸 뒤로 옮김
 - 문자가 일치하지 않으므로 표에서 3번째 문자(C)의 값을 0으로 함

중복되는 문자에
레전 정보를 저장.

A	B	C	A	B	D
A	B	C	A	B	D

A	B	C	A	B	D
-	0				

A	B	C	A	B	D
A	B	C	A	B	D

A	B	C	A	B	D
-	0	0			



07-3 KMP 법

◆ KMP법 알아보기 - (3)

◆ KMP법 - (3)

- 패턴을 1칸 뒤로 옮기면 "AB"가 일치
 - 1. 패턴의 4번째 문자 'A'까지 일치한다면 아래에 위치한 패턴을 1칸 옮긴 다음 "A"를 건너뛰고 2번째 문자부터 검사할 수 있음
 - 2. 패턴의 5번째 문자 'B'까지 일치한다면 아래에 위치한 패턴을 1칸 옮긴 다음 "AB"를 건너뛰고 3번째 문자부터 검사할 수 있음
 - 따라서 표에서 두 문자의 값을 1, 2로 할 수 있음
- 아래에 위치한 패턴을 2칸 뒤로 옮김
 - 문자가 일치하지 않으므로 표에서 패턴의 마지막 문자 'D'의 값을 0으로 함

AB C A B D

AB C A B D

AB C A B D

AB C A B D

A	B	C	A	B	D
-	0	0	1	2	

A	B	C	A	B	D
-	0	0	1	2	0



07-3 KMP 법

ABCABD

◆ KMP법 알아보기 - (4)

◆ 실습 7-13 - (1)

- KMP법을 사용해 문자열을 검색하는 함수를 작성한 프로그램
- kmp_match 함수가 받는 인수와 반환값은 브루트-포스법의 bf_match 함수와 같음
- 1 에서 다시 시작 값의 표를 만들

Do it! 실습 7-13

• 완성 파일 chap07/kmp_match.c

```

01  /*--- KMP법으로 문자열을 검색 ---*/
02  int kmp_match (const char txt[], const char pat[])
03  {
04      int pt = 1;          // txt 커서
05      int pp = 0;          // pat 커서
06      int skip[1024];      // 건너뛰기 표
07
08      skip[pt] = 0;
09      while(pat[pt] != '\0') {
10          if(pat[pt] == pat[pp])
11              skip[++pt] = ++pp;
12          else if(pp == 0)
13              skip[++pt] = pp;
14          else

```

Handwritten notes for the code:

pat[pt], pp=0

0	1	2	3
A	B	A	B

pat[pt] | A | B | A | B

0	1	2	3	4
0	0	0	1	2

1 표 만들기

```

15      pp = skip[pp];
16  }

```

① pt=2, pp=0

② skip[pt]=++pp

skip[3]=1

pt=3, pp=1

③ skip[pt]=++pp
skip[4]=2, pt=4, pp=2



07-3 KMP 법

◆ KMP법 알아보기 - (5)

◆ 실습 7-13 - (2)

- 2에서 검색을 수행
- KMP법에서 텍스트를 스캔하는 커서 pt는 다시 뒤로 돌아오지 않음
- KMP법은 브루트-포스법보다는 복잡하고, 다음 절에서 공부할 보이어-무어법과는 성능이 같거나 좋지 않아 실제 프로그램에서는 거의 사용하지 않음

```
15     pp = skip[pp];
16 }
17
18 pt = pp = 0;
19 while(txt[pt] != '\0' && pat[pp] != '\0') {
20     if(txt[pt] == pat[pp]) {
21         pt++; pp++;
22     } else if(pp == 0)
23         pt++;
24     else
25         pp = skip[pp];
26 }
27 if(pat[pp] == '\0')
28     return pt - pp;
29
30 return -1;
31 }
```

2 검색

```

01  /*--- KMP법으로 문자열을 검색 ---*/
02  int kmp_match (const char txt[], const char pat[])
03  {
04      int pt = 1;      // txt 커서
05      int pp = 0;      // pat 커서
06      int skip[1024];  // 건너뛰기 표
07
08      skip[pt] = 0;
09      while(pat[pt] != '\0') {
10          if(pat[pt] == pat[pp]) - (a)
11              skip[++pt] = ++pp;
12          else if(pp == 0) - (b)
13              skip[++pt] = pp;
14          else - (c)
15
16      }
17      pp = skip[pp];
18  }

```

pat[] =

0	1	2	3	4	5
A	B	C	A	B	D

 (pp) → 건너뛰기 표 만들기

0	1	2	3	4	5
A	B	C	A	B	D

 (pt)

pt=1, pp=0

skip[] =

0	1	2	3	4	5	6
	0	0	0	1	2	0

① if (a) n
 (b) $\rightarrow skip[+pt] = pp$
 skip[2] = 0, $\therefore pt=2, pp=0$

② if (a) n
 (b) $\rightarrow skip[+pt] = pp$
 skip[3] = 0, $\therefore pt=3, pp=0$

③ if (a) $\rightarrow skip[+pt] = ++pp$
 skip[4] = 1; $\therefore pt=4, pp=1$

④ if (a) $\rightarrow skip[+pt] = ++pp$
 skip[5] = 2, $\therefore pt=5, pp=2$

⑤ if (a) n
 (b) n
 (c) $pp = skip[pp]$
 = skip[2]; $\therefore pp=0$

⑥ if (a) n
 (b) $\rightarrow skip[+pt] = pp$
 skip[6] = 0



07-4 보이어 무어법

◆ 보이어-무어법 살펴보기 - (1)

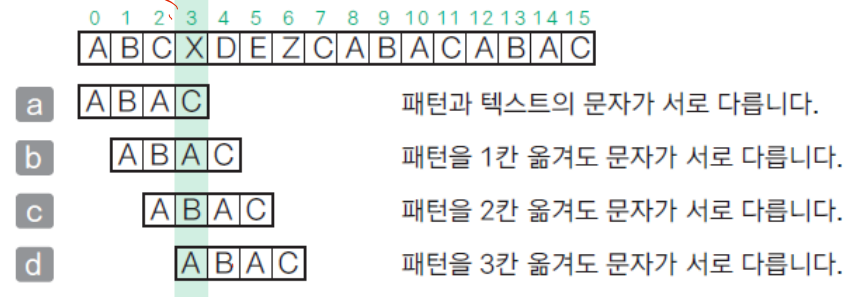
◆ 보이어-무어법

- KMP법보다 효율이 더 좋음
- 이 알고리즘은 패턴의 마지막 문자부터 앞쪽으로 검사를 진행하면서 일치하지 않는 문자가 있으면 미리 준비한 표에 따라 패턴을 옮길 크기를 정함

◆ 그림 7-13

- a 처럼 텍스트와 패턴의 첫 번째 문자를 위, 아래로 겹치고 패턴의 마지막 문자 'C'를 검사
- 텍스트의 'X'는 패턴에 아예 없는 문자이기 때문에 b ~ d 처럼 패턴을 1~3칸 옮겨도 문자열 "ABCX"와 패턴 안의 문자는 일치하지 않는다는 것을 알 수 있음

X 문자는 패턴안에 서로
일치하는 문자가 없기 때문에
패턴의 길이만큼 건너뛰기로
생각함



[그림 7-13] 패턴의 마지막 문자가 다른 경우



07-4 보이어 무어법

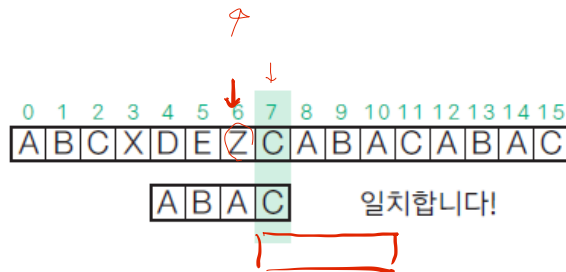
◆ 보이어-무어법 살펴보기 - (2)

◆ 그림 7-14

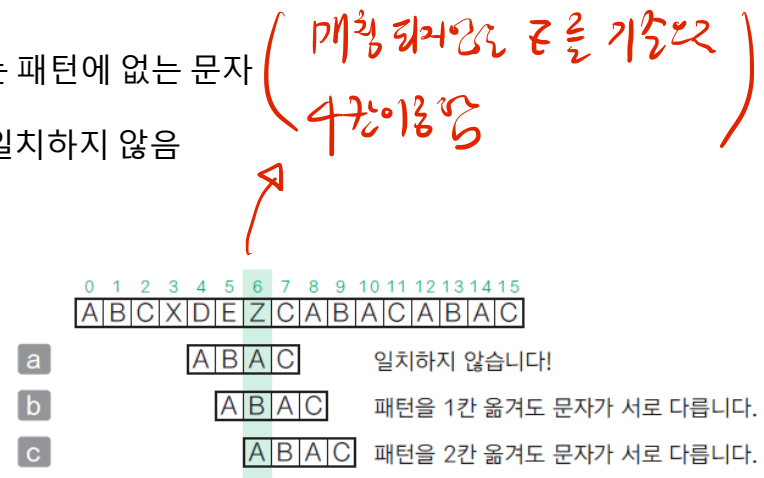
- 텍스트 안에서 패턴에 들어 있지 않은 문자를 찾으면 해당 위치까지의 문자는 건너뛸 수 있음
- 이 방법을 사용하면 **b** ~ **d**의 비교는 생략하고 패턴을 단숨에 4칸 뒤로 옮겨 그림 7-14의 상태가 됨
- 이 상태는 패턴의 마지막 문자 'C'와 텍스트의 'C'가 일치하기 때문에 패턴을 1칸 앞의 문자 'A'로 옮길 수 있음

◆ 그림 7-15

- 패턴의 문자 'A'는 텍스트의 'Z'와 다르기도 하지만 텍스트의 'Z'는 패턴에 없는 문자
- 따라서 **b** **c**처럼 패턴을 1~2칸 옮긴다고 하더라도 패턴과 일치하지 않음



[그림 7-14] 패턴의 마지막 문자가 일치하는 경우



[그림 7-15] 패턴과 텍스트의 문자가 다른 경우



07-4 보이어 무어법

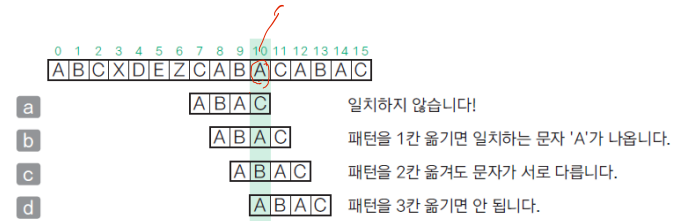
◆ 보이어-무어법 살펴보기 - (3)

◆ 그림 7-16

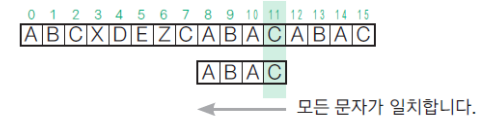
- 패턴의 길이를 n 이라고 하면 현재 검사하고 있는 텍스트의 문자 위치로부터 '다음에 검사할 패턴의 마지막 문자 위치'가 n 만큼 떨어질 수 있도록 패턴을 옮기면 됨
- 검사하고 있는 텍스트의 위치(6)로부터 4만큼 떨어진 위치(10)에서 검사를 시작하기 위해 패턴을 3칸 옮김
- 텍스트의 'A'와 패턴의 마지막 문자 'C'를 비교 a
- 문자 'A'는 패턴의 1, 3번째 인덱스에 들어 있으므로 b 와 같이 패턴의 뒤쪽에 위치한 'A'가 텍스트와 위, 아래로 겹치도록 패턴을 1칸만 옮김

◆ 그림 7-17

- 다시 패턴의 마지막 위치에서 순서대로 문자를 비교하면 모두 일치하기 때문에 검색 성공



[그림 7-16] 패턴과 텍스트의 문자가 다른 경우



[그림 7-17] 검색에 성공한 경우



07-4 보이어 무어법

◆ 보이어-무어법 살펴보기 - (4)

◆ 건너뛰기 표

- 그런데 보이어-무어 알고리즘도 각각의 문자를 만났을 때 패턴을 옮길 크기를 저장할 표(건너뛰기 표)를 미리 만들어야 함
- 패턴 문자열의 길이가 n 일 때 옮길 크기는 아래와 같은 방법으로 결정
- 패턴에 들어 있지 않은 문자를 만난 경우
 - 1. 패턴을 옮길 크기는 n
- 패턴에 들어 있는 문자를 만난 경우
 - 1. 마지막에 나오는 위치의 인덱스가 k 이면 패턴을 옮길 크기는 $n - k - 1$
 - 2. 같은 문자가 패턴 안에 중복해서 들어 있지 않다면("ABAC"의 'C'는 패턴 안에 1개만 들어 있음) 패턴을 옮길 크기는 n

Handwritten notes illustrating the calculation of the jump size:

For the pattern "ABAC" (length $n=4$):

- For 'A' at index 0: $4 - 1 - 1 = 2$
- For 'B' at index 1: $4 - 2 - 1 = 1$
- For 'A' at index 2: $4 - 2 - 1 = 1$
- For 'C' at index 3: $4 - 3 - 1 = 0$ (labeled as "마지막 문자" - last character)

텍스트 ... "ABCXDEZCABAC" 패턴 ... "ABAC"

A	B	C	D	E	F	G	H	I	J	K	L	M
1	2	4	4	4	4	4	4	4	4	4	4	4
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	4	4	4	4	4	4	4	4	4	4	4	4

[그림 7-18] 건너뛰기 표



07-4 보이어 무어법

◆보이어-무어법 살펴보기 - (5)

◆실습 7-14

- bm_match 함수가 받는 인수와 반환값은 지금까지 살펴본 두 개 함수와 같음
- 패턴에 존재할 수 있는 모든 문자의 옮길 크기를 계산하고 저장해야 하기 때문에 건너뛰기 표(배열 skip)의 요소 개수는 $\text{UCHAR_MAX} + 1$



07-4 보이어 무어법

Do it! 실습 7-14

• 완성 파일 chap07/bm_match.c

```
01 // 보이어-무어법으로 문자열을 검색하는 프로그램
02 #include <stdio.h>
03 #include <string.h>
04 #include <limits.h>
05
06 /*--- 보이어-무어법으로 문자열을 검색하는 함수 ---*/
07 int bm_match (const char txt[], const char pat[])
08 {
09     int pt; // txt 커서
10     int pp; // pat 커서
11     int txt_len = strlen(txt); // txt 문자 개수
12     int pat_len = strlen(pat); // pat 문자 개수
13     int skip[ UCHAR_MAX + 1 ]; // 건너뛰기 표
14     for(pt = 0; pt <= UCHAR_MAX; pt++) // 건너뛰기 표 만들기
15         skip[pt] = pat_len;
16     for(pt = 0; pt < pat_len - 1; pt++)
17         skip[pat[pt]] = pat_len - pt - 1;
18
19     while(pt < txt_len) { // pt == pat_len - 1
20         pp = pat_len - 1; // pat의 마지막 문자부터 검사
21         while(txt[pt] == pat[pp]) {
22             if(pp == 0)
23                 return pt;
24             pp--;
25             pt--;
26         }
27         pt += (skip[txt[pt]] > pat_len - pp) ? skip[txt[pt]] : pat_len - pp;
28     }
29     return -1;
30 }
```

실행 결과

보이어-무어법
텍스트: ABABCFGHHA
패턴: ABC
3번째 문자부터 match합니다.



07-4 보이어 무어법

◆보이어-무어법 살펴보기 - (6)

◆실습 7-14

- 보이어-무어법을 사용한 프로그램

```
32 int main (void)
33 {
34     char s1[256];           // 텍스트
35     char s2[256];           // 패턴
36     puts("보이어-무어법");
37     printf("텍스트: ");
38     scanf("%s", s1);
39     printf("패턴: ");
40     scanf("%s", s2);
```

```
41     int idx = bm_match(s1, s2);    // 문자열 s1에서 문자열 s2를 보이어-무어법을 사용해 검색
42     if(idx == -1)
43         puts("텍스트에 패턴이 없습니다.");
44     else
45         printf("%d번째 문자부터 match합니다.\n", idx + 1);
46
47     return 0;
48 }
49
```



07-4 보이어 무어법

◆ strstr 함수 알아보기 - (1)

◆ strstr 함수

- C 언어는 문자열을 검색하는 표준 라이브러리 strstr 함수를 제공
- strchr, strrchr 함수와 마찬가지로 찾은 위치의 인덱스가 아닌 그 문자에 대한 포인터를 반환한다는 점을 주의
- 검색에 실패하면 널 포인터를 반환

strstr 함수	
헤더	#include <string.h>
형식	char *strstr(const char *s1, const char *s2);
해설	s1이 가리키는 문자열에서 s2가 가리키는 문자열과 일치하는(널 문자를 포함하지 않는) 문자열을 찾습니다. 가장 앞쪽에 나오는 문자열을 찾습니다.
반환값	찾아낸 문자열에 대한 포인터(첫 번째 문자에 대한 포인터)를 반환합니다. 찾지 못하면 널 포인터를 반환합니다. s2가 길이가 0인 문자열이면 s1을 반환합니다.



07-4 보이어 무어법

◆ strstr 함수 알아보기 - (2)

◆ 실습 7-15

- strstr 함수를 사용하여 문자열을 검색하는 프로그램
- 이 프로그램은 printf 함수를 적절히 사용하여 같은 문자가 위, 아래로 겹쳐지도록 출력

Do it! 실습 7-15

• 완성 파일 chap07/strstr_test.c

```
01 // strstr 함수를 사용한 프로그램
02 #include <stdio.h>
03 #include <string.h>
04
05 int main (void)
06 {
07     char s1[256], s2[256];
08     puts("strstr 함수");
09     printf("텍스트: ");
10     scanf("%s", s1);
11     printf("패턴: ");
12     scanf("%s", s2);
13     char *p = strstr(s1, s2); // 문자열 s1에서 문자열 s2를 검색
14     if(p == NULL)
15         printf("텍스트에 패턴이 없습니다. \n");
16     else {
17         int ofs = p - s1;
18         printf("\n%s\n", s1);
19         printf("%*s|\n", ofs, "");
20         printf("%*s%s\n", ofs, "", s2);
21     }
22
23     return 0;
24 }
```

실행 결과

strstr 함수
텍스트: ABABCDEFGHA
패턴: ABC
ABABCDEFGHA
|
ABC